第三次课程作业

作业内容：将 `Assignment3` 中提供的线程相关示例代码逐个运行，并结合运行结果进行说明。

测试环境（Windows）

项目	说明
操作系统	Windows 11
.NET SDK	`9.0.301`
目标框架	`net9.0`

---

1 运行记录

1.1 `00 创建线程`

输出内容：

```
Starting...
1
2
3
4
5
6
7
8
9
Starting...
1
2
3
4
5
6
7
8
9
```

说明：

- 主线程与新建线程都执行 `PrintNumbers`，所以会出现两组 `Starting... + 1..9`。
- 两个线程并发执行，实际输出顺序可能交错；本次运行中刚好先后完整输出两段。

1.2 `01 暂停线程`

输出内容：

```
Starting...
Starting...
1
2
3
4
5
6
7
8
9
1
2
3
4
5
6
7
8
9
```

说明：

- 一个线程执行 `PrintNumbersWithDelay`（每次循环 `Sleep(2s)`），另一个线程立即执行 `PrintNumbers`。
- 因为延时线程更慢，主线程先快速打印完 `1..9`，随后延时线程再逐步输出 `1..9`。

1.3 `02 等待线程结束`

输出内容：

```
Starting program...
Starting...
1
2
3
4
5
6
7
8
9
Thread completed
```

说明：

- 主线程在 `t.Start()` 后调用 `t.Join()`，因此会阻塞等待子线程结束。
- 只有子线程输出完 `1..9` 后，主线程才继续输出 `Thread completed`。

1.4 `03 终止线程`

输出内容（节选）：

```
warning SYSLIB0006: Thread.Abort() is not supported...
Starting program...
Starting...
1
2
Unhandled exception. System.PlatformNotSupportedException: Thread abort is not
supported on this platform.
...
```

说明：

- 在 `net9.0` 下，`Thread.Abort()` 与 `Thread.ResetAbort()` 已过时且不受支持。
- 运行到 `t.Abort()` 时主线程抛出 `PlatformNotSupportedException`，程序异常结束。
- 这也说明该示例属于 .NET Framework 时代写法，在现代 .NET 中应改为协作式取消（如 `CancellationToken`）。

1.5 `04 线程运行状态`

输出内容（节选）：

```
Starting program...
Unstarted
Running
Running
...
Starting...
Running
1
2
Unhandled exception. System.PlatformNotSupportedException: Thread abort is not
supported on this platform.
```

说明：

- 线程 `t` 在 `Start()` 前状态是 `Unstarted`，启动后在循环中多次观察到 `Running`。
- 线程函数开始后也会打印自身状态（`Running`），与主线程观察一致。
- 示例后半段同样调用了 `Thread.Abort()`，在 .NET 9 下触发 `PlatformNotSupportedException` 并中断程序。

1.6 `05 线程优先级`

输出内容：

```
Affinity = 1
ThreadOne(Highest): 27,790,485,909
ThreadTwo(Lowest) :      3,435,925

Affinity = 3
ThreadOne(Highest):  4,002,887,859
ThreadTwo(Lowest) :    149,450,767

Affinity = 7
ThreadOne(Highest):  3,882,129,578
ThreadTwo(Lowest) :  3,507,191,965
```

说明：

- `ProcessorAffinity=1`（二进制 `001`，单核）时，高优先级线程几乎“碾压”低优先级线程，差距最大。
- `ProcessorAffinity=3`（二进制 `011`，双核）时，低优先级线程开始有更多执行机会，但仍明显落后。
- `ProcessorAffinity=7`（二进制 `111`，三核）时，两者计数接近，说明核心数越多，优先级对吞吐差距的影响通常越小。
- 原代码未 `Join` 两个计数线程，所以输出中会先看到 `Main End.`，随后才打印两个线程的统计结果。

1.7 `06 前后台线程`

输出内容（节选）：

```
ForegroundThread prints 0
BackgroundThread prints 0
...
ForegroundThread prints 9
BackgroundThread prints 10
...
BackgroundThread prints 19
```

说明：

- 当前代码把 `threadTwo.IsBackground = false`，所以两个线程都属于前台线程。
- 即使 `Main` 很快结束，进程仍会等待两个线程都执行完（因此能看到 `BackgroundThread` 打印到 `19`）。
- 若改成 `true`，后台线程可能在前台线程结束后被进程直接终止，输出会提前中断。

1.8 `07 向线程传递参数`

输出内容（节选）：

```
TestSome x: 100 y :500 z: 2022
ThreadOne prints 1 ... 10
--------------------------
ThreadTwo prints 1 ... 8
--------------------------
ThreadThree prints 1 ... 12
--------------------------
20
21
```

说明：

- 示例演示了三种线程传参方式：实例方法捕获构造参数、`ParameterizedThreadStart`、lambda 闭包。
- `TestSome(y:500, x:100)` 展示了命名参数 + 可选参数（`z` 使用默认值 2022）。
- `ref` 示例中两个线程共享同一个变量 `i`，先后打印 `20`、`21`，体现共享状态会被并发修改。

1.9 `08 使用Monitor lock`

输出内容：

```
Incorrect counter
Total count: 178627
--------------------------
Correct counter with Monitor
Total count: 300000
Correct counter with lock
Total count: 300000
```

说明：

- 三个线程各加 `100000` 次，理论总数应为 `300000`。
- 不加锁的 `Counter` 发生竞态，结果明显小于 `300000`。
- `Monitor` 和 `lock` 两种同步方案都能得到正确结果，验证了临界区互斥保护的作用。

1.10 `09 死锁`

输出内容：

```
Monitor.TryEnter returning false after a specified timeout is elapsed
Timeout acquiring a resource!
----------------------------------
This will be a deadlock!
```

说明：

- 第一段使用 `Monitor.TryEnter(lock1, 5s)`，超时后输出 `Timeout acquiring a resource!`，避免了永久阻塞。
- 第二段改为直接 `lock(lock1)`，与另一个线程形成锁顺序反转，进入死锁。
- 程序在打印 `This will be a deadlock!` 后不再继续，本次运行为完成记录已手动停止进程。

1.11 `10` 线程函数要处理异常

输出内容：

```
Starting a faulty thread...
Exception handled: Boom!
----------------------
Starting a faulty thread...
Unhandled exception. System.Exception: Boom!
```

说明：

- 第一段线程函数内部自行 `try-catch`，异常被正确处理，主流程继续。
- 第二段线程函数未捕获异常，异常不会被主线程外层 `try-catch` 接住，最终导致进程崩溃。
- 结论：线程入口应尽量自行处理异常并记录日志，避免整个进程非预期终止。

1.12 `11` 使用互锁函数

输出内容：

```
Incorrect counter
Total count: 196857
--------------------------
Correct counter
Total count: 300000
```

说明：

- 普通 `++` 在并发下不是原子操作，结果偏小。
- 使用 `Interlocked.Increment` 后，每次加法原子化，最终总数稳定为理论值 `300000`。
- 这个示例展示了“简单计数场景”下 `Interlocked` 相比显式锁更轻量的优势。

1.13 `12` 使用生产者消费者队列

输出内容（节选）：

```
[消费者] 队列为空，进入等待...
[生产者] 生产 1，当前队列数量：1
[消费者] 消费 1，当前队列数量：0
...
[生产者] 队列已满，进入等待...
[消费者] 消费 4，当前队列数量：2
[生产者] 生产 7，当前队列数量：3
...
[消费者] 消费 10，当前队列数量：0
演示结束。
```

说明（通俗版）：

- 把队列想成一个最多放 3 件货的“小仓库”：生产者放货，消费者拿货。
- 条件变量的作用是：**条件不满足就先睡觉，条件改变后再被叫醒继续干活。**

- 当队列空时，消费者执行 `Monitor.Wait` 进入等待；当生产者放入新数据后，`Monitor.PulseAll` 唤醒等待线程。
- 当队列满时，生产者也会 `Wait`；等消费者取走数据腾出空间后，再被唤醒继续生产。
- `Wait` 会暂时释放锁，避免别人一直拿不到锁；被唤醒后会重新抢锁并继续执行。
- 代码使用 `while` 检查条件（不是 `if`），是为了防止“被唤醒时条件又不成立”的情况。

1.14 `13 使用线程局部存储`

输出内容（本次运行）：

```
counterPerThread: 50582367
counterPerThread: 56592483
```

说明：

- 当前代码里 `[ThreadStatic]` 被注释掉了，因此 `counterPerThread` 实际是**共享静态变量**。
- 两个线程各执行 `50000000` 次自增，但因为并发竞争，最终打印的是两个竞争中的中间值，且通常不同。
- 若启用 `[ThreadStatic]`，每个线程会有自己的 `counterPerThread` 副本，预期会更接近各自独立计数结果。

1.15 `14 一些语言特性`

输出内容：

```
Can ignore Console.
Compare True
Compare2 True
SomeFeature myName myName
SomeFeature 18 19
age: 18
age:
age:
```

说明：

- 示例验证了静态 `using`、表达式方法体、表达式属性、自动属性初始化器、空值传播运算符等语法点。
- `sf = null` 后，`sf?.Age` 返回 `null`，因此输出空值（`Nullable<int>` 无值时显示为空）。

---

2 总结

本次已按顺序逐个运行 `Assignment3` 中可检测到的全部示例代码，并在每次运行后立即补充报告。核心结论如下：

1. 基础线程操作（创建、`Sleep`、`Join`）行为符合预期。
2. `Thread.Abort` 在现代 .NET（`net9.0`）不再可用，应改为协作式取消。
3. 并发共享数据若无同步会出现竞态；`lock/Monitor/Interlocked` 均可在不同场景提供正确性保障。

4. 死锁案例清晰展示了“锁顺序不一致”的风险，`TryEnter` 超时是实用的防护手段。
5. 线程异常应在线程入口处自行处理，否则容易导致进程直接崩溃。